

ADR-002 — Redis pour double usage : Idempotence des paiements et tokens MFA

Champ	Valeur
Statut	Accepté
Date	2026-01-20
Auteurs	Équipe CanBankX

Contexte

Deux problèmes distincts nécessitaient un stockage temporaire avec expiration automatique (TTL) :

1. **Idempotence des paiements** (`payment-service`) : les paiements bancaires doivent être exacts-
once. Un réseau instable peut provoquer des retransmissions. Sans idempotence, un virement peut
être débité deux fois.
2. **Tokens de challenge MFA** (`identity-service`) : lors du login en deux facteurs, un token de
challenge est émis entre l'étape "login" et l'étape "vérification OTP". Ce token doit expirer
automatiquement (5 minutes) et ne pas persister en base de données.

Les options considérées pour l'idempotence :

- **Base de données uniquement** : vérifier la clé d'idempotence dans MySQL avant chaque insertion →
lent, génère du lock contention.
- **Cache Redis + fallback DB** : vérifier Redis d'abord (μ s), fallback sur MySQL si Redis est vidé →
recommandé.
- **Cache en mémoire JVM** : efficace mais ne fonctionne pas en multi-instances.

Décision

Redis est utilisé pour les deux cas d'usage avec des TTL distincts :

Usage	Service	Clé Redis	TTL
Challenge MFA	<code>identity-service</code>	<code>mfa:challenge:{token}</code>	5 minutes
Idempotence paiement	<code>payment-service</code>	<code>payment:idem:{key}</code>	24 heures

La stratégie d'idempotence utilise un **double filet de sécurité** :

1. Vérification Redis (rapide, ~1 ms)
2. Fallback sur requête MySQL `findByIdempotencyKey` si Redis est vide (ex : redémarrage Redis)

Conséquences

Positives

- **Performance mesurée** : les tests de charge k6 ont enregistré **9 603 cache hits** sur 4 minutes à 50 VUs, évitant autant de tentatives d'écriture en base de données.
- **Expiration automatique** : les tokens MFA expirés sont nettoyés sans tâche planifiée.
- **Résistance aux redémarrages** : le fallback DB garantit l'idempotence même après un restart Redis.
- **Instance partagée** : un seul conteneur Redis sert les deux services, simplifiant le déploiement.

Négatives / Risques

- **Redis est un SPOF** : si Redis devient indisponible, les deux services dégradent (plus de MFA, plus de garantie d'idempotence rapide). Mitigation : le fallback DB maintient la correction mais augmente la latence.
- **Consistance Redis-DB** : une fenêtre de race condition existe entre la vérification Redis et l'insertion DB. Elle est gérée par la contrainte **UNIQUE** sur **idempotencyKey** en base de données, qui rejette les doublons au niveau SQL.